



UNIVERSIDAD DE MÁLAGA
Dpto. Lenguajes y Ciencias de la Computación
E.T.S.I. Informática

Fundamentos de la Programación
Examen 2ª Convocatoria Ordinaria

04/09/19

Apellidos, Nombre:

Titulación:

Grupo:

Código PC usado:

NOTAS PARA LA REALIZACIÓN DEL EXAMEN:

- La solución se almacenará en la carpeta **EXAMENSEPFP**, dentro de **Documentos**. Si la carpeta ya existe, debe borrarse todo su contenido. En otro caso, debe crearse.
- Los nombres de los ficheros con la solución para los ejercicios 1, 2, 3 y 4 serán **ejercicio1.cpp**, **ejercicio2.cpp**, **ejercicio3.cpp** y **ejercicio4.cpp**, respectivamente.
- Al inicio del contenido de cada fichero deberá aparecer un comentario con **el nombre del alumno, titulación, grupo y código del equipo** que se está utilizando (cada dato en una línea diferente).
- **Debe consultarse** el documento “**Obligaciones y Recomendaciones Estilo de Programación**”, disponible en el Campus Virtual de la asignatura, con objeto de tener en cuenta los puntos allí señalados en las soluciones a los ejercicios.
- Una vez terminado el examen, se subirán los ficheros ***.cpp** a la tarea creada en el **campus virtual** para ello.
- **No está permitido:**
 - Utilizar documentación electrónica o impresa.
 - Intercambiar documentación con otros compañeros.

(1 pto) 1.- Dado un tipo *array* de 10 números enteros, se debe desarrollar una **función** que devuelva el índice del *array* donde se encuentra el primer elemento (aquel con menor valor de índice) que sea **mayor o igual** que la **media** de los valores almacenados en el array. Se valorará la eficiencia del algoritmo desarrollado.

Por ejemplo, para los números {3, 1, 4, 0, 7, 2, 5, 9, 8, 6} devuelve el índice 4 (que se corresponde con el índice del elemento con valor 7).

Importante: sólo se completará el código del subprograma `buscar_mayig_media` en el fichero `ejercicio1.cpp` proporcionado en el campus virtual. Puedes añadir más subprogramas si lo consideras necesario. No se debe modificar el resto del código proporcionado. La puntuación de este problema será de 1 punto sólo en el caso de que el subprograma funcione correctamente y sea eficiente. En otro caso la puntuación será de 0 puntos.

La ejecución del código suministrado (tras diseñar el procedimiento solicitado) será, para los datos de entrada 3 1 4 0 7 2 5 9 8 6:

Introduce 10 números: 3 1 4 0 7 2 5 9 8 6

El primer elemento mayor o igual que la media se encuentra en 4

(2 ptos) 2.- Una matriz guarda una imagen de M x N píxeles que tiene dibujada una circunferencia (que no tiene por qué estar centrada). Diseña una **función** `diametro`, que recibiendo como parámetro de entrada una variable conteniendo una imagen, devuelva un número entero que indique el diámetro de la circunferencia dibujada en ella. Se puede suponer que el color de fondo tiene el carácter ‘ ’ y el de la línea de la circunferencia el carácter ‘*’, que siempre existe una única circunferencia y no hay ningún tipo de error.

Ejemplo 1 (circunferencia1)

	0	1	2	3	4	5	6	7	8	9	10	11
0												
1			*	*								
2		*			*							
3	*					*						
4	*					*						
5		*			*							
6			*	*								

Ejemplo 2 (circunferencia2)

	0	1	2	3	4	5	6	7	8	9	10	11
0												
1												
2												
3												
4										*		
5									*		*	
6										*		

Importante: se completará el código de la función `diametro` en el fichero `ejercicio2.cpp` proporcionado en el campus virtual. Puedes añadir más procedimientos o funciones si lo estimas necesario. No se debe modificar el resto del código proporcionado.

La ejecución del código suministrado (tras diseñar la función) será:

```
Diametro circunferencial es: 4
Diametro circunferencia2 es: 1
```

(3.5 ptos) 3.- Diseña un algoritmo que lea de teclado un texto y muestre por pantalla un listado de todas las longitudes distintas de palabras del texto, indicando para cada longitud su número de ocurrencias. **En la salida no habrá longitudes repetidas.**

Ejemplo:

Entrada:

Introduzca un texto (FIN para terminar):

CREO QUE IREMOS A MI CASA PRIMERO Y QUE DESPUES IREMOS A LA CASA QUE QUIERAS FIN

Salida:

Longitudes Ocurrencias

4 3

3 3

6 2

1 3

2 2

7 3

NOTAS:

- El texto contiene un número indefinido de palabras.
- El texto termina con la palabra FIN.
- Cada palabra tiene un número indefinido pero limitado de caracteres (todos alfabéticos mayúsculas).
- El carácter separador de palabras es el espacio en blanco.
- En el texto habrá un número máximo MAX_LONG_DIST (una constante) de palabras de longitudes distintas.

(3.5 ptos) 4.- Se desea informatizar la gestión de la venta de entradas para una sala de cine, que consta de cinco (5) filas, cada una de ellas con siete (7) asientos, de tal forma que cada asiento puede estar en estado **reservado** o **libre**, representados por las letras ('x') y ('o') respectivamente en la siguiente figura:

	0	1	2	3	4	5	6
0	x	x	o	o	x	o	o
1	o	o	o	o	o	o	o
2	o	x	o	o	o	x	o
3	x	x	x	x	o	o	o
4	o	o	o	o	o	o	o

Complete el código en el fichero `ejercicio4.cpp` proporcionado en el campus virtual definiendo los siguientes subprogramas. Puedes añadir más subprogramas si lo consideras necesario. No se debe modificar el resto del código proporcionado.

- `void inicializar(SalaCine& sc)`
crea e inicializa la sala de cine `sc` recibida como parámetro con todos los asientos en estado libre.

- `void mostrar(const SalaCine& sc)`
muestra en pantalla el estado de los asientos de la sala de cine `sc` recibida como parámetro, según el formato del siguiente ejemplo:

```

      0 1 2 3 4 5 6
-----
0:  x x o o x o o
1:  o o o o o o o
2:  o x o o o x o
3:  x x x x o o o
4:  o o o o o o o

```

- `void comprar_ticket_consecutivo(SalaCine& sc, int fila_1, int fila_2, int n, bool& ok, int& fil_sel, int& col_sel)`

dada la sala de cine `sc`, un rango de filas (desde `fila_1` hasta `fila_2`, ambas inclusive), y un número (`n`) de asientos solicitados recibidos como parámetros, si los parámetros son correctos (`fila_1` menor o igual a `fila_2`, ambos dentro del rango correcto, y `n` mayor que cero), entonces busca si hay algún bloque de (`n`) asientos libres **consecutivos** en dicho rango de filas, y si lo encuentra, entonces seleccionará el primer bloque (menor fila y menor columna) de `n` asientos libres que se encuentre dentro del rango especificado, marcará todos los asientos seleccionados al estado reservado, el parámetro de salida `ok` tomará el valor `true`, y devolverá en los parámetros de salida `fil_sel` y `col_sel` el valor de la fila y columna respectivamente donde se encuentra el primer asiento del bloque seleccionado.

En caso contrario, entonces el parámetro de salida `ok` tomará el valor `false`.

Por ejemplo, para la figura del subprograma `mostrar`, para el rango de filas `{2, 3}` y una solicitud de (3) asientos consecutivos, el parámetro `fil_sel` tomará el valor 2 y `col_sel` tomará el valor 2, y la sala de cine quedará en el siguiente estado de reservas (en negrita las nuevas reservas):

```

      0 1 2 3 4 5 6
-----
0:  x x o o x o o
1:  o o o o o o o
2:  o x x x x x o
3:  x x x x o o o
4:  o o o o o o o

```

- `void cancelar_ticket(SalaCine& sc, int fila, int columna, bool& ok)`
dada la sala de cine `sc`, y un número de fila y columna que especifica un determinado asiento en la sala de cine `sc`, recibidos como parámetros, si los datos recibidos como parámetros son correctos y el asiento especificado está reservado en la sala `sc`, entonces **cancela** la reserva de dicho asiento en la sala `sc`, que pasará a estar **libre**, y el parámetro de salida `ok` tomará el valor `true`. En otro caso, el parámetro de salida `ok` tomará el valor `false`.